Sinhgad Technical Education Society's
Sinhgad Institute of Business Administration and Research
Kondhwa (Bk) Pune-411048
Master of Computer Application
Affiliated to Savitribai Phule Pune University and Approved by AICTE, New Delhi
Sinhgad Institutes

A PROJECT REPORT

ON

Face Recognition based Attendance Management System

SAVITRIBAI PHULE PUNE UNIVERSITY

In Partial Fulfillment of

MASTERS OF COMPUTER APPLICATION

By

Omkar Dushyant Patil (68)

Babu Mahadev Lahade(32)

Sinhgad Institutes

SINHGAD INSTITUTE OF BUSINESS ADMINISTRATION AND RESEARCH,
KONDHWA, PUNE – 411048

2024-26

Sinhgad Technical Education Society's
Sinhgad Institute of Business Administration and Research
Kondhwa (Bk) Pune-411048
Master of Computer Application
Affiliated to Savitribai Phule Pune University and Approved by AICTE, New Delhi

Sinhgad Institutes

SINHGAD TECHNICAL EDUCATION SOCIETY'S
SINHGAD INSTITUTE OF BUSINESS ADMINISTRATION & RESEARCH
(Approved by AICTE, Recognized by Government of Maharashtra,
Affiliated to Savitribai Phule Pune University, Accredited by NAAC)
Near PMC Octroi Post, Kondhwa - Saswad Road, Kondhwa (Bk), Pune - 411048
Phone : +91 20 26934543/26933635/20 26933633 Email : directormca_sibar@sinhgad.edu Website : www.sinhgad.edu

Prof. M. N. Navale
M.E. (Elect.), MIE, MBA
FOUNDER PRESIDENT

Dr. (Mrs.) Sunanda M. Navale
B.A, M.P.M., Ph. D.
FOUNDER SECRETARY

Dr. Netra Patil
MCA, Ph. D. (Computer Mgmt.)
DIRECTOR

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled '**Face Recognition based Attendance Management System'** submitted to the Department of MCA, **Sinhgad Institute of Business Administration and Research** in partial fulfillment of the requirement for the award of the degree of MASTER OF COMPUTER APPLICATIONS (Affiliated to Savitribai Phule Pune University), is an original work carried out by

Omkar D. Patil
Roll No: - 68
Babu M. Lahade
Roll No: -32

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this Organization or to any other University/Organization for the fulfillment of the requirement of any course of study.

Signature of the Student :

Name of the student : **Omkar D. Patil**

Signature of the Student :

Name of the student : **Babu M. Lahade**

Signature of the Guide :

Name and Designation of the Guide : **Dr. Priya Chaudhari**

Signature of Director-MCA :

Director-SIBAR MCA : **Dr. Netra Patil**
Date :

CERTIFICATE OF APPROVAL

This is to certify that the Project Report entitled '**Face Recognition based Attendance Management System' s**ubmitted to the Department of MCA, **Sinhgad Institute of Business Administration and Research** in partial fulfillment of the requirement for the award of the Degree of MASTER OF COMPUTER APPLICATIONS (**Affiliated to Savitribai Phule Pune University**) is an original work carried out by

Omkar Dushyant Patil **Roll No…68…**

Babu Mahadev Lahade **Roll No…32…**

The matter embodied in this project is a genuine work done by the student and has been certified by the following internal and external examiners deputed by Savitribai Phule Pune University.

Internal Examiner

ACKNOWLEDGEMENT

We find great pleasure in expressing our deep sense of attitude towards all those who have made this possible for use to complete this project with success.

We would like to thank our Director, SIBAR-MCA **Dr. Netra Patil** and our Project Guide **Dr. Priya Chaudhari** and other staff members of college for cooperating us for Project and providing us valuable experience. Our classmate and friends need mention here for being so co-operative, understanding and helping us every time during our project work.

Student Name **Student Sign**
Omkar Dushyant Patil
Babu Mahadev Lahade

MCA – Sem II

ITC21 – Mini Project

Project Guidelines: Application Development Project

Chapter No		Details
1		**Introduction**
	1.1	Abstract
	1.2	Existing System and Need for System
	1.3	Scope of System
	1.4	Operating Environment - Hardware and Software
	1.5	Brief Description of Technology Used 1.5.1 Operating systems used (Windows or Unix) 1.5.2 RDBMS/No Sql used to build database (mysql/ oracle, Teradata,etc.)
2		**Proposed System**
	2.1	Study of Similar Systems (If required research paper can be included)
	2.2	Feasibility Study
	2.3	Objectives of Proposed System
	2.4	Users of System
3		**Analysis and Design**
	3.1	System Requirements (Functional and Non-Functional requirements)
	3.2	Entity Relationship Diagram (ERD)
	3.3	Table Structure
	3.4	Use Case Diagrams
	3.5	Class Diagram
	3.6	Activity Diagram
	3.7	Deployment Diagram
	3.8	Module Hierarchy Diagram
	3.9	Sample Input and Output Screens (Screens must have valid data. All reports must have at-least 5 valid records.)
4		**Coding**
	4.1	Algorithms
	4.2	Complete Code of Project
5		**Testing**
	5.1	Test Strategy
	5.2	Unit Test Plan
	5.3	Acceptance Test Plan
	5.4	Test Case / Test Script
	5.5	Defect report / Test Log
6		**Limitations of Proposed System**
7		**Proposed Enhancements**
8		**Conclusion**
9		**Bibliography**
10		**Publication (if any)**
11		**User Manual** (All screens with proper description/purpose Details about validations related to data to be entered.)

Attendance Management System Documentation

1. Introduction

1.1 Abstract

The Attendance Management System is designed to automate the process of taking student attendance using face recognition technology. It ensures accuracy, reduces manual effort, and provides real-time attendance records.

1.2 Existing System and Need for System

Existing System: Traditional attendance systems use manual roll calls or RFID-based systems, which are time-consuming and prone to proxy attendance. Need for System: The proposed system eliminates proxy attendance, reduces human intervention, and automates record-keeping for easy retrieval.

1.3 Scope of System

- Real-time face recognition-based attendance.
- Secure storage in MongoDB and CSV files.
- Admin and student dashboards for attendance tracking.
- Filtering by subject and time duration.
- Data visualization using pie charts**.**

1.4 Operating Environment - Hardware and Software

Hardware: Webcam, System with at least 8GB RAM, i5 Processor.
Software: Windows OS, Python, Flask, OpenCV, MongoDB, Pandas, Matplotlib, Bootstrap.

1.5 Brief Description of Technology Used

- Operating System: Windows/Linux
- Database: MongoDB (NoSQL)
- Libraries & Frameworks: Flask, OpenCV, Pandas, Matplotlib, Bootstrap

2. Proposed System

2.1 Study of Similar Systems

Similar systems use RFID or biometric scanners, but they require additional hardware. Face recognition eliminates this requirement.

2.2 Feasibility Study

- Technical Feasibility: Uses widely available hardware and open-source libraries.
- Economic Feasibility: Cost-effective as it eliminates the need for additional hardware.
- Operational Feasibility: Easy to integrate with existing school/university systems.

2.3 Objectives of Proposed System

- Ensure accurate attendance marking.
- Automate record storage and retrieval.
- Provide statistical analysis through dashboards.

2.4 Users of System

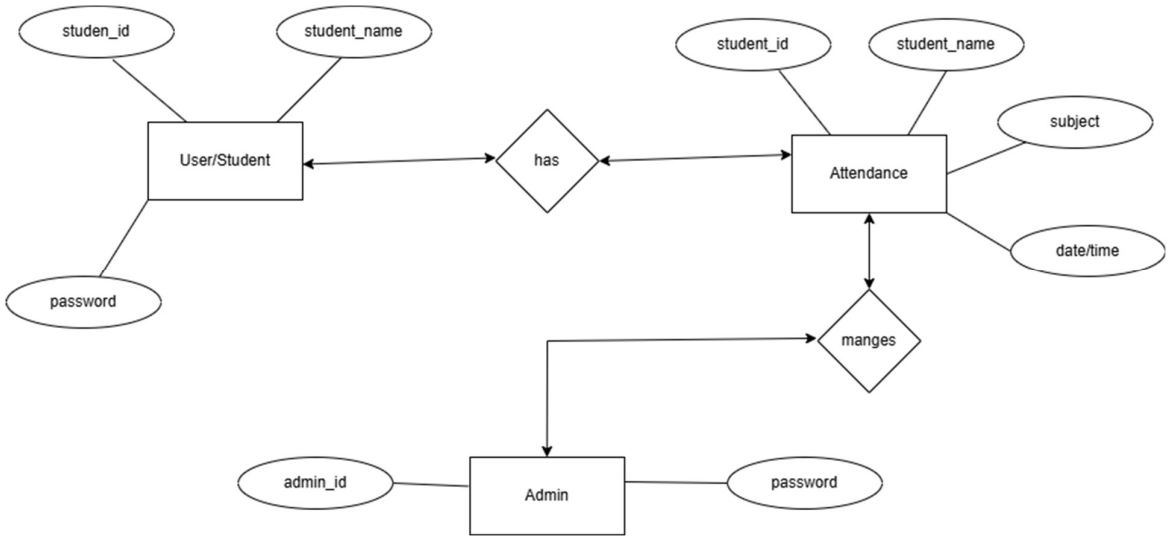
- **Admin:** Manages student data and monitors attendance.
- **Student:** Views personal attendance records.

3. Analysis and Design

3.1 System Requirements

- **Functional Requirements:** Face recognition for attendance, admin dashboard, student dashboard, filtering options.
- **Non-Functional Requirements:** Security, scalability, real-time processing.

3.2 Entity Relationship Diagram (ERD)



3.3 Table Structure

- **Attendance Collection:**

Name	Type	Null	default
Student ID	Int(64)	no	
Name	String	no	
Subject	String	no	
Timestamp	Date-time	no	

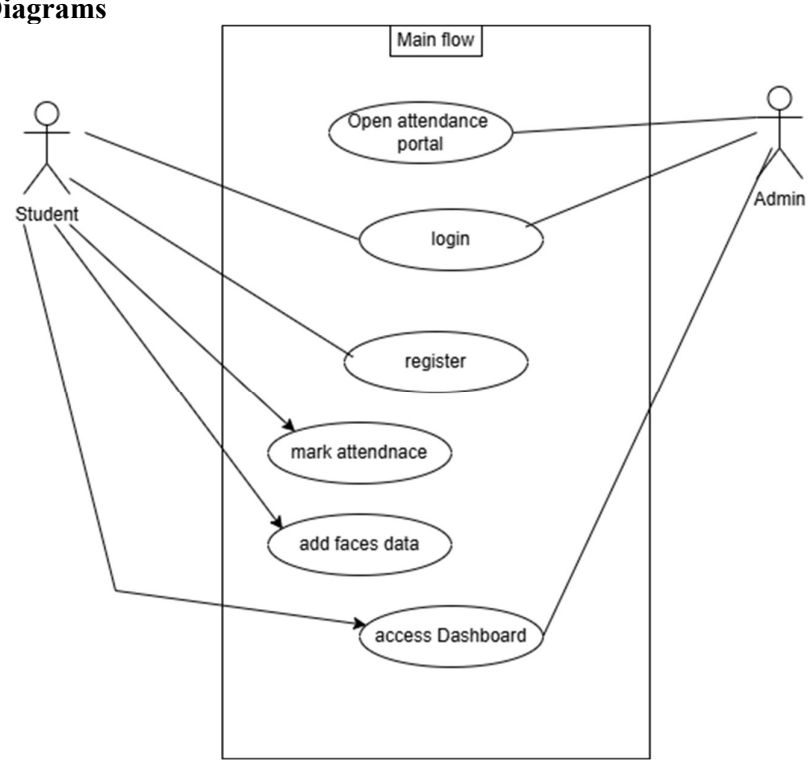
- **Student_info Collection:**

Name	Type	Null	default
Student ID	Int(64)	no	
Name	String	no	
Password	String	no	

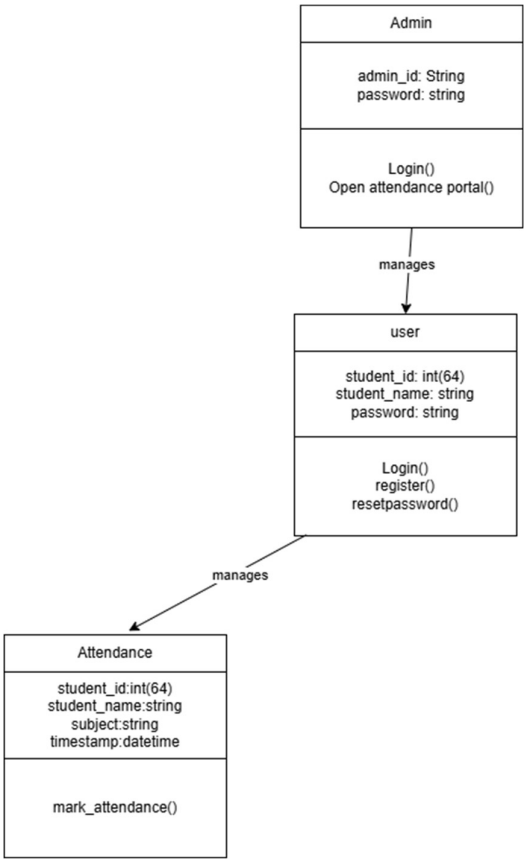
- **Admin_info Collection:**

Name	Type	Null	default
Admin name	string	no	
Password	String	no	

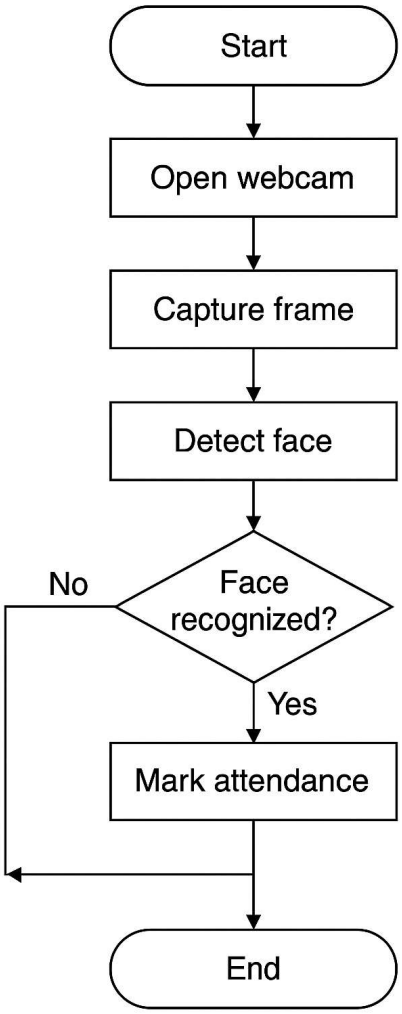
3.4 Use Case Diagrams



3.5 Class Diagram

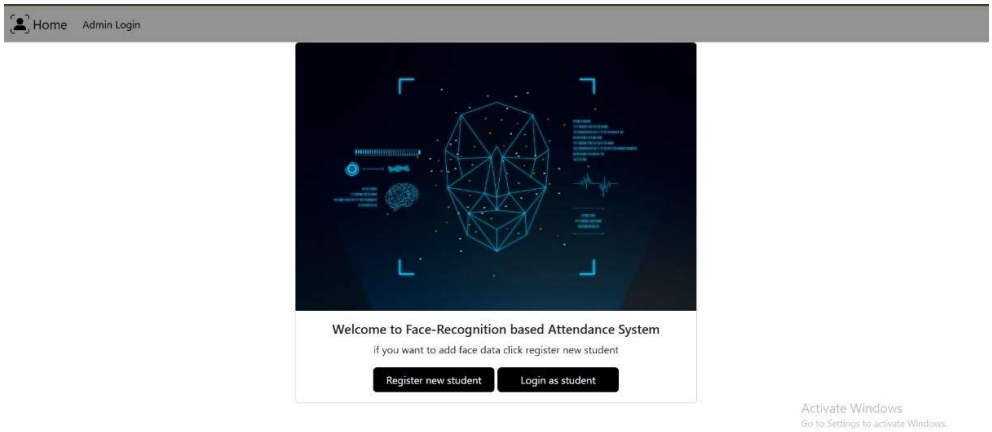


3.6 Activity Diagram

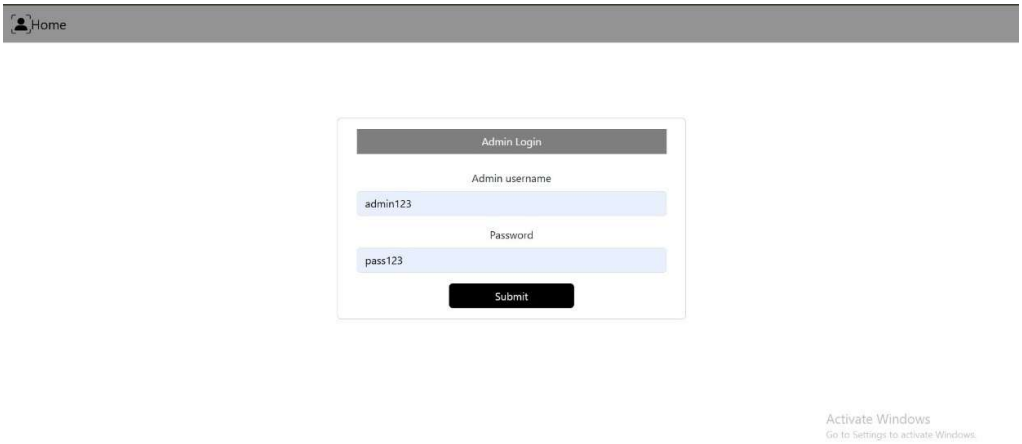


3.9 Sample Input and Output Screens

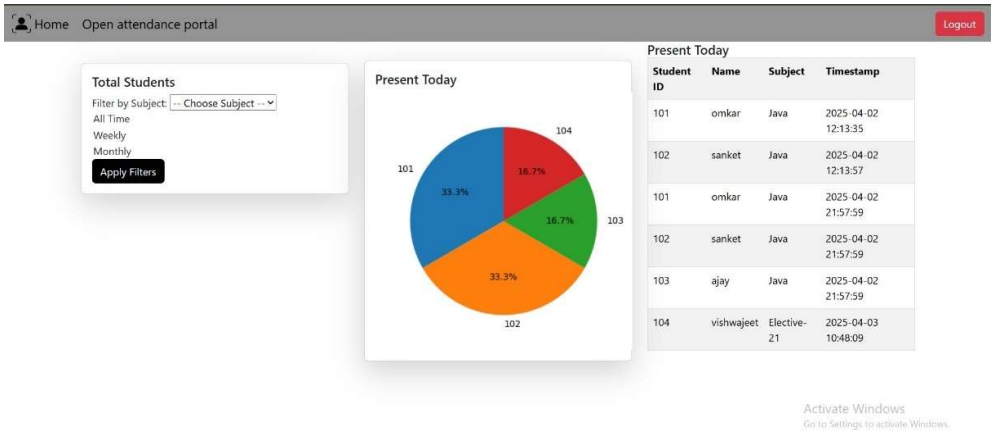
3.9.1 Index page



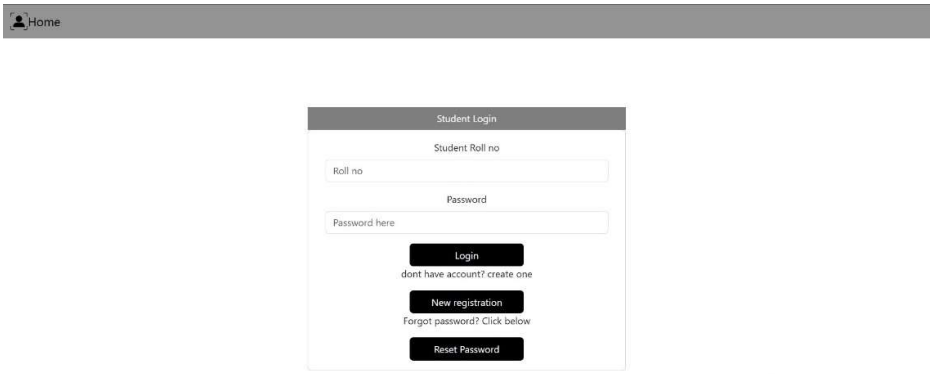
3.9.2 Admin login



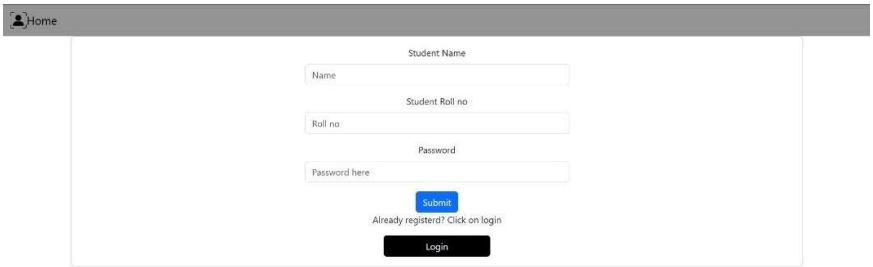
3.9.3 Admin dashboard



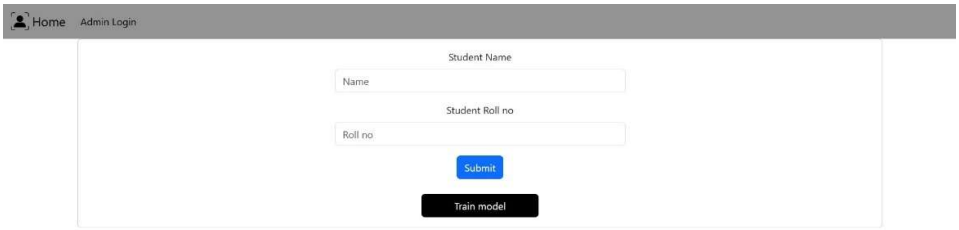
3.9.4 Student Login



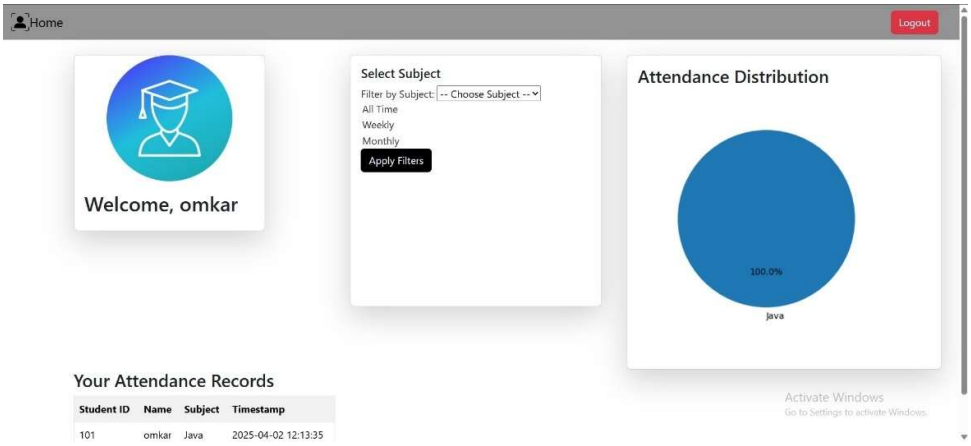
3.9.5 Student registration



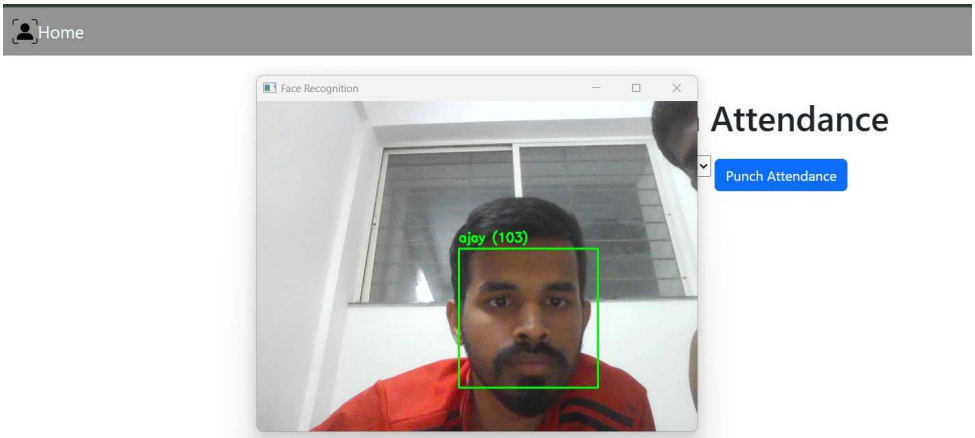
3.9.6 Student Face registration



3.9.8 Student Dashboard



3.9.10 Real time recognition



4. Coding

4.1 Algorithms

- Face recognition using OpenCV and SVM.
- Filtering attendance records based on time and subject.

4.2 Complete Code of Project

```
from flask import Flask,render_template,redirect,request,flash,url_for,session
import cv2
import pickle
import numpy as np
import os
import pandas as pd
from sklearn.svm import SVC
from datetime import datetime,timedelta
from sklearn.preprocessing import StandardScaler
```

```
from flask_pymongo import PyMongo
from werkzeug.security import generate_password_hash,check_password_hash
import matplotlib.pyplot as plt
import io
import base64

app = Flask(__name__)
app.secret_key = "Omkar"

#database_connectivity
app.config["MONGO_URI"] = "mongodb+srv://root:toor@omkar.wuznt.mongodb.net/College"
mongo = PyMongo(app)

@app.route('/')
def home():
    # password="pass123"
    # hashed_password = generate_password_hash(password)
    # mongo.db.Admin_info.insert_one({"Admin_id": "admin123", "password": hashed_password})
    return render_template('index.html')

#admin login
@app.route('/adminlogin', methods=['GET', 'POST'])
def adminlogin():
    if request.method == "POST":
        Admin_id = request.form.get('Admin_id')  # Ensure correct input retrieval
        password = request.form.get('password')

        if not Admin_id or not password:
            flash("Both fields are required!", "danger")
            return render_template('adminlogin.html')

        # Fetch admin from DB
        Admin = mongo.db.Admin_info.find_one({"Admin_id": Admin_id})

        if Admin and check_password_hash(Admin["password"], password):
            session["Admin_id"] = str(Admin["Admin_id"])  # Ensure session stores string
            return redirect(url_for('admindashboard'))  # Use url_for() for proper routing

        flash("Invalid Admin ID or Password", "danger")
        return render_template('adminlogin.html')  # Ensure flash message is shown

    return render_template('adminlogin.html')  # Render login page for GET request

#admindashboard
@app.route('/admindashboard',methods=['GET','POST'])
def admindashboard():
    subject = request.args.get('subject')  # Get subject filter
    time_filter = request.args.get('time')  # Weekly or Monthly

    # Fetch attendance data
    query = {}
    if subject:
        query["subject"] = subject  # Filter by subject

    attendance_data = list(mongo.db.Attendance.find(query))

    # Convert timestamps to datetime
    for record in attendance_data:
        record["timestamp"] = datetime.strptime(record["timestamp"], "%Y-%m-%d %H:%M:%S")

    # Filter by time range
    if time_filter == "weekly":
```

```
    start_date = datetime.now() - timedelta(days=7)
  elif time_filter == "monthly":
    start_date = datetime.now() - timedelta(days=30)
  else:
    start_date = None  # Show all data

  if start_date:
    attendance_data = [record for record in attendance_data if record["timestamp"] >= start_date]

  # Convert to DataFrame for easier manipulation
  df = pd.DataFrame(attendance_data)

  # Attendance count per student
  attendance_counts = df["student_id"].value_counts()

  # Generate Pie Chart
  fig, ax = plt.subplots(figsize=(4,4))
  ax.pie(attendance_counts, labels=attendance_counts.index, autopct='%1.1f%%', startangle=90)
  ax.axis('equal')

  ax.set_position([0.1, 0.1, 0.8, 0.8])
  # Convert plot to base64 image for HTML
  img = io.BytesIO()
  plt.savefig(img, format='png')
  img.seek(0)
  plot_url = base64.b64encode(img.getvalue()).decode()
  return render_template('admindashboard.html', students=attendance_data, plot_url=plot_url)

#adminlogout
@app.route('/adminlogout')
def adminlogout():
  session.pop("Admin_id", None)  # Remove student_id from session
  flash(" ✅ You have been logged out.", "success")
  return redirect(url_for('adminlogin'))  # Redirect to login page

#student login
@app.route('/studentlogin', methods=['GET', 'POST'])
def studentlogin():
  if request.method == "POST":
    student_id = request.form.get('student_id')  # Fix tuple issue
    password = request.form.get('password')

    if not student_id or not password:
      flash("Both fields are required!", "danger")
      return render_template('studentlogin.html')

    # Fetch student from DB
    student = mongo.db.Student_info.find_one({"student_id": student_id})

    if student and check_password_hash(student["password"], password):
      session["student_id"] = str(student["student_id"])  # Ensure string
      session["student_name"] = student["student_name"]
      return redirect('/studentdashboard')

    flash("Invalid Student ID or Password", "danger")

  return render_template('studentlogin.html')

#reset password
@app.route('/resetpassword', methods=['GET', 'POST'])
def resetpassword():
  if request.method == 'POST':
```

```
    student_id = request.form.get('student_id')
    new_password = request.form.get('new_password')

    if not student_id or not new_password:
        flash("Student ID and new password are required!", "danger")
        return redirect(url_for('resetpassword'))

    student = mongo.db.Student_info.find_one({"student_id": student_id})

    if student:
        hashed_pass = generate_password_hash(new_password)  # Hash new password

        mongo.db.Student_info.update_one(
            {"student_id": student_id},
            {"$set": {"password": hashed_pass}}
        )
        flash("✅ Password changed successfully! Please log in.", "success")
        return redirect(url_for('studentlogin'))  # Redirect to login

    flash("❌ Student ID not found!", "danger")
    return redirect(url_for('resetpassword'))  # Redirect back

return render_template('resetpassword.html')  # Render password reset page

#studentlogout
@app.route('/studentlogout')
def logout():
    session.pop("student_id", None)  # Remove student_id from session
    session.pop("student_name", None)  # Remove student_name if stored
    flash("✅ You have been logged out.", "success")
    return redirect(url_for('studentlogin'))  # Redirect to login page

#studentdashboard
@app.route('/studentdashboard',methods=['GET','POST'])
def studentdashboard():
    if 'student_id' not in session:
        flash("Please log in to access your dashboard", "danger")
        return redirect(url_for('login'))  # Redirect if not logged in

    student_id = session['student_id']  # Get logged-in student's ID
    subject = request.args.get('subject')  # Get subject filter
    time_filter = request.args.get('time')  # Weekly or Monthly filter

    # Fetch attendance for the logged-in student
    query = {"student_id": student_id}
    if subject:
        query["subject"] = subject  # Filter by subject

    attendance_data = list(mongo.db.Attendance.find(query))

    # Convert timestamps to datetime
    for record in attendance_data:
        record["timestamp"] = datetime.strptime(record["timestamp"], "%Y-%m-%d %H:%M:%S")

    # Filter by time range
    if time_filter == "weekly":
        start_date = datetime.now() - timedelta(days=7)
    elif time_filter == "monthly":
        start_date = datetime.now() - timedelta(days=30)
    else:
        start_date = None  # Show all data
```

```
    if start_date:
        attendance_data = [record for record in attendance_data if record["timestamp"] >= start_date]

    # Convert to DataFrame for easier manipulation
    df = pd.DataFrame(attendance_data)

    # Attendance count per subject
    attendance_counts = df["subject"].value_counts()

    # Generate Pie Chart
    fig, ax = plt.subplots(figsize=(4, 4))
    ax.pie(attendance_counts, labels=attendance_counts.index, autopct='%1.1f%%', startangle=90)
    ax.axis('equal')

    # Convert plot to base64 image for HTML
    img = io.BytesIO()
    plt.savefig(img, format='png')
    img.seek(0)
    plot_url = base64.b64encode(img.getvalue()).decode()

    return render_template('studentdashboard.html', attendance=attendance_data, plot_url=plot_url)


@app.route('/goto', methods=['GET','POST'])
def goto():
    return render_template('mark_attendance.html')

#studengt logiregistration
@app.route('/studentregistration',methods=['GET','POST'])
def studentregistration():
    if request.method == "POST":
        data = {
            "student_id": request.form['student_id'],
            "student_name": request.form['student_name'],
            "password": generate_password_hash(request.form['password'])  # Hash password
        }
        mongo.db.Student_info.insert_one(data)
    return render_template('studentregistration.html',)


#add new student
@app.route('/add_student', methods=['GET', 'POST'])
def add_student():
    face_cascade = cv2.CascadeClassifier('data/haarcascade_frontalface_default.xml')

    # Load existing face data
    faces_data = {}

    if os.path.exists("data/faces_data.pkl"):
        with open("data/faces_data.pkl", "rb") as f:
            faces_data = pickle.load(f)

    if request.method == 'POST':  # Start face capture only when form is submitted
        sname = request.form['sname']
        sid = request.form['sid']

        if not sid or not sname:
            return render_template("add_student.html")

        # Initialize dictionary if student ID is new
        if sid not in faces_data:
```

```
        faces_data[sid] = {"name": sname, "faces": []}

    video = cv2.VideoCapture(0)  # Open camera
    count = 0

    while count < 100:  # Collect 100 faces
        ret, frame = video.read()
        if not ret:
            continue

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)

        for (x, y, w, h) in faces:
            face_crop = frame[y:y+h, x:x+w]
            resized_face = cv2.resize(face_crop, (50, 50)).flatten()

            faces_data[sid]["faces"].append(resized_face)  # Store with Student ID
            count += 1

            # Draw rectangle around detected face
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.putText(frame, f"Collected: {count}/100", (x, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

        cv2.imshow("Collecting Faces", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    # Release camera and close OpenCV windows
    video.release()
    cv2.destroyAllWindows()

    # Save updated face data
    with open("data/faces_data.pkl", "wb") as f:
        pickle.dump(faces_data, f)

    print(f"Face dataset for {sname} ({sid}) saved successfully!")
    return render_template('add_student.html', message="Face data saved successfully!")

return render_template('add_student.html')  # Load form initially

#goto

#marking attendance
@app.route('/mark_attendance', methods=['POST'])
def mark_attendance():
    subject = request.form.get('subject')  # Get subject from form

    if not subject:
        flash("Please select a subject!", "danger")
        return redirect(url_for('goto'))

    today_date = datetime.now().strftime("%d-%m-%Y")
    attendance_file = f"Attendance/attendance_{today_date}.csv"

    # Ensure attendance file exists
    if not os.path.exists(attendance_file):
        df = pd.DataFrame(columns=["Student ID", "Name", "Subject", "Timestamp"])
        df.to_csv(attendance_file, index=False)

    # Load trained model and scaler
```

```
with open("data/svm_model.pkl", "rb") as f:
    svm_model = pickle.load(f)

with open("data/scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# Load face data
with open("data/faces_data.pkl", "rb") as f:
    faces_data = pickle.load(f)

student_names = {sid: data["name"] for sid, data in faces_data.items()}
face_cascade = cv2.CascadeClassifier("data/haarcascade_frontalface_default.xml")

video = cv2.VideoCapture(0)
recognized_students = {}

while True:
    ret, frame = video.read()
    if not ret:
        continue

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)

    for (x, y, w, h) in faces:
        face_crop = frame[y:y+h, x:x+w]
        resized_face = cv2.resize(face_crop, (50, 50)).flatten().reshape(1, -1)
        resized_face = scaler.transform(resized_face)

        sid_pred = str(svm_model.predict(resized_face)[0])
        student_name = student_names.get(sid_pred, "Unknown")

        if student_name != "Unknown":
            recognized_students[sid_pred] = student_name

        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, f"{student_name} ({sid_pred})", (x, y-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

    cv2.imshow("Face Recognition", frame)

    if cv2.waitKey(1) & 0xFF == ord('o') and recognized_students:
        break
    if cv2.waitKey(1) & 0xFF == ord('q'):
        video.release()
        cv2.destroyAllWindows()
        return redirect(url_for('goto'))

video.release()
cv2.destroyAllWindows()

timestamp = datetime.now()
df = pd.read_csv(attendance_file)

for sid, name in recognized_students.items():
    last_attendance = mongo.db.Attendance.find_one(
        {"student_id": sid, "subject": subject},
        sort=[("timestamp", -1)]
    )

    if last_attendance:
        last_time = datetime.strptime(last_attendance["timestamp"], "%Y-%m-%d %H:%M:%S")
```

```
        if timestamp - last_time < timedelta(hours=1):
            flash(f"❌ {name} ({sid}) already marked attendance for {subject} in the last hour!", "danger")
            continue

        # Save to MongoDB
        attendance_record = {
            "student_id": sid,
            "name": name,
            "subject": subject,
            "timestamp": timestamp.strftime("%Y-%m-%d %H:%M:%S")
        }
        mongo.db.Attendance.insert_one(attendance_record)

        # Save to CSV
        new_entry = pd.DataFrame([[sid, name, subject, timestamp.strftime("%Y-%m-%d %H:%M:%S")]],
                                 columns=["Student ID", "Name", "Subject", "Timestamp"])
        df = pd.concat([df, new_entry], ignore_index=True)
        print(f"✅ Attendance marked: {name} ({sid}) for {subject} at {timestamp}")

    df.to_csv(attendance_file, index=False)

    flash("Attendance marked successfully!", "success")
    return redirect(url_for('goto'))


#training model
@app.route('/trainmodel',methods=["GET","POST"])
def trainmodel():
    faces_path = "data/faces_data.pkl"
    model_path = "data/svm_model.pkl"
    scaler_path = "data/scaler.pkl"

    # Check if face data file exists
    if not os.path.exists(faces_path):
        raise FileNotFoundError("❌ Face data not found! Please add students first.")

    # Load dataset
    with open(faces_path, "rb") as f:
        faces_data = pickle.load(f)

    # Extract features (faces) and labels (student IDs)
    FACES = []
    LABELS = []

    for sid, data in faces_data.items():
        for face in data["faces"]:
            FACES.append(face)
            LABELS.append(sid)  # Store Student ID as label

    # Convert to NumPy array
    FACES = np.array(FACES)
    LABELS = np.array(LABELS)

    # Ensure at least two unique students exist for training
    unique_labels = np.unique(LABELS)
    if len(unique_labels) < 2:
        raise ValueError("❌ Dataset must contain at least two different students for training!")

    # Normalize features
    scaler = StandardScaler()
    FACES = scaler.fit_transform(FACES)
```

```
    # Train SVM model
    svm_model = SVC(kernel='linear', probability=True)
    svm_model.fit(FACES, LABELS)

    # Save model and scaler
    with open(model_path, "wb") as f:
        pickle.dump(svm_model, f)

    with open(scaler_path, "wb") as f:
        pickle.dump(scaler, f)

    print("✅ Model trained and saved successfully!")
    flash("Model trained successfully","success")
    return render_template('trainmodel.html')


if __name__ == "__main__":
    app.run(debug=True,port=5000)
```

5. Testing

Category	Test Case	Expected Outcome
Authentication	Valid admin credentials	Admin successfully logs in.
	Invalid admin credentials	Login fails with an error message.
	Empty username/password fields	Validation error displayed.
	Valid student credentials	Student logs in successfully.
	Invalid student credentials	Login fails with an error message.
Attendance Marking	Student faces camera with registered face	Attendance is marked successfully.
	Student tries to mark attendance twice within an hour	Attendance is rejected with a warning.
	Unregistered face scanned	Attendance is not recorded.
Data Storage	Attendance record added to MongoDB	The record appears in the 'Attendance' collection.
	Attendance record saved in CSV file	Data is correctly written to the file.
Filtering	Admin filters attendance by subject	Attendance list displays only selected subject data.
	Admin filters attendance by weekly/monthly range	Data is filtered correctly.
Dashboard (Admin)	Admin views attendance statistics	Pie chart displays correct attendance distribution.
	Admin selects a subject to filter dashboard	Chart updates accordingly.
Dashboard (Student)	Student views their own attendance data	Only the logged-in student's records are shown.
	Attendance percentage is calculated correctly	Displayed percentage matches expected values.
Error Handling	MongoDB connection failure	System displays a database error message.
	CSV file is missing or inaccessible	Attendance marking should fail gracefully.
	Multiple students mark attendance simultaneously	System handles concurrency properly.

6. Limitations of Proposed System

- Dependence on camera quality.
- Possible recognition errors under poor lighting conditions.

7. Proposed Enhancements

- Implement deep learning for improved recognition.
- Add mobile application support.

8. Conclusion

The project successfully automates attendance marking using face recognition and provides analytical insights through dashboards.

9. Bibliography

Youtube: https://youtu.be/JKSU7lbZ3ZE?si=fsDBoWyjmnv5QV7E10.
Github: https://github.com/amlanmohanty1/face-recognition-attendance-management-system-with-PowerBI-dashboard

10. User Manual

- Login: Enter credentials to access dashboard.
- Mark Attendance: Face recognition-based automated marking.
- View Attendance: Filter records based on subject and date.
- Admin Panel: Manage student records, view statistics,open attendance portal.